GitHub Workflow and Contribution Guide

This document outlines the standard process for contributing to projects using GitHub.
It covers branching strategy, commit message conventions, version tagging, CI/CD integration, and best practices for maintaining code quality.
Following this workflow ensures smooth collaboration, clear version tracking, and stable deployments.

1. Overview

The main branch is the primary branch and always contains the latest stable version of the project.
Any changes pushed to main will trigger deployment if CI/CD is configured.
All development work is done in separate branches and reviewed via Pull Requests (PRs) before merging into main.

2. Contribution Workflow

Step 1 – Clone the Repository

```
git clone <repository-url>
cd <project-folder>
```

Step 2 – Create a New Branch from main

1. Always branch from the latest main:
2. git checkout main
3. git pull origin main
4. git checkout -b feature/your-feature-name

Branch Naming Guidelines:

1. feature/ for new functionality (e.g., feature/add-auth)
2. fix/ for bug fixes (e.g., fix/header-alignment)
3. update/ for dependency or code updates (e.g., update/api-endpoints)
4. chore/ for maintenance or minor changes (e.g., chore/update-readme)

Step 3 – Make Changes

1. Develop and test changes in your branch.
2. Run linting and formatting before committing:
3. Identify linting issues.
4. Apply agreed-upon formatting automatically.

Step 4 – Write Good Commit Messages

Use the Conventional Commits format:
Examples:

```
feat: implement user login
fix: correct header display on mobile
docs: update setup instructions
chore: clean up unused styles
```

Best Practices:

Make atomic commits (one logical change per commit).
Keep the subject ≤ 72 characters.

Step 5 – Update CHANGELOG.md
Summarize changes under the correct version or in an Unreleased section if no new tag is yet created.

Step 6 – Push Your Branch and Create a Pull Request

git push origin feature/your-feature-name
On GitHub, create a Pull Request:

Step 7 – Review and Approval
All changes must be reviewed before merging.
Address reviewer comments promptly.
PRs must pass all CI checks before merge.

Step 8 – Merge and Deploy
Once approved:
Merge to main.

3. Best Practices
1. Keep branches and PRs focused on a single change.
2. Pull updates from main frequently.
3. Test thoroughly before pushing.
4. Use descriptive commit messages and PR titles.
5. Avoid large PRs; smaller ones are easier to review.

4. CI/CD and Branch Protection
1. Protect main (and develop if used) from direct pushes.
2. Require PR reviews and passing CI before merging.
3. Use CODEOWNERS to assign reviewers automatically.
4. Enforce commit conventions via automated checks.

5. Hotfixes
For urgent fixes:
git checkout -b hotfix/issue-description main
Apply the fix, test, and open a PR to main.

Best Practices
- Keep branches and PRs focused and easy to review.
- Pull updates from `main` frequently to stay current.
- Test thoroughly before pushing.
- Use descriptive commit messages and PR titles.
Following this workflow ensures consistency, traceability, and a reliable release process for the your current project.